# 深度学习导论　作业二（HW2）：PyTorch 基础

戴士博　学号：2025012128

2026 年 11 月 2 日

本报告对应的代码与数据均为本人本地运行所得；Question 4 的 notebook 已全部执行并保存输出。

## 1　运行环境与配置

本作业使用隔离的 conda 环境 `pytorch-learning`，仅使用 CPU。notebook 开头固定了随机种子，并按照题目要求固定了批大小、优化器、学习率与训练轮数。具体信息见表 1。

**表 1：** 运行环境与实验配置

| 项目 | 值 |
|---|---|
| Python | 3.11.15（Anaconda，独立 conda 环境 `pytorch-learning`） |
| PyTorch | 2.12.1+cpu |
| TorchVision | 0.27.1+cpu |
| 运行设备 | CPU（无 GPU），`torch.set_num_threads(2)` |
| MNIST 数据 | 本地 IDX 文件，经 `prepare_mnist.py` 放至 `data/MNIST/raw`，无需联网 |
| 随机种子 | 2026（模型初始化、训练集 shuffle 生成器均固定） |
| 批大小 / 优化器 / 学习率 / 轮数 | 64 / SGD / 0.1 / 5 |

由于固定了种子且全程使用 CPU，重复运行得到的数值可以复现；但题目也说明，跨 PyTorch 版本或跨设备不保证完全一致。

## 2　概念题（Question 1，1.00 分）

### 2.1　1.1 CrossEntropyLoss 的输入与目标

**答：不需要先对 logits 做 softmax。** `nn.CrossEntropyLoss` 的输入应为形状 $(N, C)$ 的原始 logits 和形状 $(N,)$ 的整数类别目标；它在内部把 `log_softmax` 与负对数似然（NLL）合并计算，等价于“先 softmax 再取 $-\log p_y$”，但数值更稳定。若先手动 softmax 再传入，相当于重复归一化，会让梯度变小甚至饱和，是错误用法。

目标的 dtype 必须是整数类型 `torch.long`（即 `int64`），不能是浮点或 one-hot；合法标签范围是 $[0, C-1]$，本题 $C = 10$，即标签取 $0, 1, \ldots, 9$。超出范围会触发索引错误或未定义行为。

## 2.2 1.2 model.eval() 是否会关闭梯度计算

**答：不会。**`model.eval()` 只是把模块切换到“推理模式”，改变的是某些层的前向行为：Dropout 停止随机丢弃、BatchNorm 改用训练时累计的 running mean/var 而不是当前 batch 统计量；不影响 autograd，前向仍会构建计算图、梯度仍可计算。

`torch.no_grad()` 才是关闭梯度记录（不构建计算图）的上下文管理器，用于节省显存与计算；两者的目的不同：`eval()` 控制“层怎么算”，`no_grad()` 控制“要不要记录梯度”。推理时通常二者一起使用：`model.eval()` + `with torch.no_grad():`。

## 2.3 1.3 标量反向传播：x.grad 与 y.grad

对题目代码，令 $x = 2$，则 $y = 3x = 6$，$\text{loss} = y^2 = 36$。由链式法则

$$\frac{\partial \text{loss}}{\partial y} = 2y = 12, \qquad \frac{\partial \text{loss}}{\partial x} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial x} = 12 \times 3 = 36.$$

因此 `x.grad` 为 `tensor(36.)`。`y.grad` 默认为 `None`：PyTorch 只为叶子张量（`is_leaf=True`，如这里的 `x`）累积 `.grad`，`y` 是中间结果、不是叶子，因此不保留梯度。

若要保留 $y$ 的梯度，应在调用 `backward()` 之前对 $y$ 调用 `y.retain_grad()`；这样反向传播后 `y.grad` 为 `tensor(12.)`。一张计算图被 `backward()` 释放后不能再次反向传播，演示必须重新做一次前向。

```
x = torch.tensor(2.0, requires_grad=True)
y = 3 * x
loss = y ** 2
loss.backward()
# x.grad = 36.0, y.grad = None, y.is_leaf = False

y2 = 3 * x      # fresh graph; x is a leaf
y2.retain_grad()
loss2 = y2 ** 2
loss2.backward()
# y2.grad = 12.0, x.grad = 36.0
```

运行结果见下图。

```
================================================================================
Part 1: the assignment snippet (only the leaf x keeps .grad)
================================================================================
x               = 2.0
y = 3 * x       = 6.0
loss = y ** 2   = 36.0
x.grad          = 36.0    (expected 36.0 = d loss / d x = 18 x)
y.grad          = None    (expected None: autograd only keeps .grad for leaves)
y.is_leaf       = False

================================================================================
Part 2: retaining the gradient of y (fresh graph, y2.retain_grad())
================================================================================
y2 = 3 * x2     = 6.0
loss2 = y2 ** 2 = 36.0
y2.grad         = 12.0    (expected 12.0 = d loss / d y = 2 y)
x2.grad         = 36.0    (expected 36.0 = d loss / d x = 18 x)
y.grad (first graph, unchanged) = None

All checks passed.
```

**图 1:** 1.3 的运行结果：`x.grad` 与 `y.grad`，以及 `retain_grad()` 的用法

### 2.4 1.4 分类器末端是 softmax 时能否用 MSE

**数学上可能，但不推荐。**若分类器末端为 softmax，输出为概率向量 $p = \mathrm{softmax}(z)$，对 one-hot 目标 $t$ 计算 $L_{\mathrm{MSE}} = \frac{1}{N}\sum_i \|p_i - t_i\|^2$ 是完全可定义、可微的复合函数，autograd 也能算出梯度，因此“可行”。

但类索引分类通常优先交叉熵：(1) 交叉熵是最大似然/最小化负对数似然，概率解释与训练目标一致；(2) 梯度性质更好：softmax+ 交叉熵对 logits 的梯度是 $p - t$，预测错得越离谱梯度越大；(3) softmax+ 交叉熵对 logits 是凸的，而 softmax+MSE 非凸，优化更困难。因此结论是“可能但不推荐”

## 3 张量操作（Question 2，1.00 分）

### 3.1 2.1 算术：A、B、C 与四种运算

按题目要求创建 float32 的 $5 \times 5$ 随机张量 $A, B$，并用 `torch.zeros_like(A)` 创建 `C`；所有运算都用 PyTorch 完成，没有使用 NumPy。关键表达式如下：

```
torch.manual_seed(2026)
A = torch.rand(5, 5, dtype=torch.float32)
B = torch.rand(5, 5, dtype=torch.float32)
C = torch.zeros_like(A)          # same shape/dtype, all zeros

A + B      # elementwise addition
A - B      # elementwise subtraction
A * B      # ELEMENTWISE (Hadamard) product
A @ B      # MATRIX product (A.matmul(B))
```

说明：`A + B`、`A - B`、`A * B` 都是逐元素运算（形状保持 $5 \times 5$）；`A @ B` 是矩阵乘法，按“行乘列再求和”计算，结果仍是 $5 \times 5$（因为两个因子都是方阵）。元素乘与矩阵乘结果完全不同，报

告中已将二者分别明确标注为 ELEMENTWISE 与 MATRIX。运行结果见图 2 与图 3。

```
================================================================================
Question 2.1: tensor arithmetic (float32, shape (5, 5))
================================================================================
A: shape=(5, 5), dtype=torch.float32
B: shape=(5, 5), dtype=torch.float32
C = torch.zeros_like(A): shape=(5, 5), dtype=torch.float32
C contains only zeros: True

A =
tensor([[0.1525, 0.2519, 0.7310, 0.0643, 0.0187],
        [0.4945, 0.7581, 0.0009, 0.6982, 0.0465],
        [0.8129, 0.1954, 0.7241, 0.6406, 0.3991],
        [0.3372, 0.9120, 0.6968, 0.5067, 0.9349],
        [0.4340, 0.7726, 0.7091, 0.5335, 0.7676]])

B =
tensor([[0.2425, 0.8866, 0.3060, 0.3328, 0.0651],
        [0.9373, 0.4193, 0.8448, 0.5726, 0.1109],
        [0.5420, 0.7902, 0.4440, 0.3822, 0.3807],
        [0.8318, 0.6251, 0.0798, 0.4043, 0.3680],
        [0.1550, 0.5816, 0.8601, 0.3990, 0.7775]])

C = torch.zeros_like(A) =
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

**图 2:** 2.1 的输入张量 $A, B$ 与零张量 $C$

```
A + B (elementwise addition) =
tensor([[0.3949, 1.1386, 1.0370, 0.3971, 0.0838],
        [1.4318, 1.1774, 0.8457, 1.2708, 0.1574],
        [1.3549, 0.9857, 1.1682, 1.0229, 0.7798],
        [1.1690, 1.5372, 0.7766, 0.9110, 1.3029],
        [0.5890, 1.3542, 1.5692, 0.9325, 1.5451]])

A - B (elementwise subtraction) =
tensor([[-0.0900, -0.6347,  0.4250, -0.2686, -0.0465],
        [-0.4428,  0.3389, -0.8439,  0.1256, -0.0645],
        [ 0.2709, -0.5948,  0.2801,  0.2584,  0.0184],
        [-0.4946,  0.2869,  0.6170,  0.1024,  0.5668],
        [ 0.2790,  0.1910, -0.1511,  0.1345, -0.0098]])

ELEMENTWISE product, A * B (NOT matrix multiplication) =
tensor([[0.0370, 0.2234, 0.2237, 0.0214, 0.0012],
        [0.4635, 0.3179, 0.0008, 0.3998, 0.0052],
        [0.4406, 0.1544, 0.3215, 0.2449, 0.1519],
        [0.2805, 0.5701, 0.0556, 0.2049, 0.3441],
        [0.0673, 0.4494, 0.6099, 0.2129, 0.5968]])

MATRIX product, A @ B (matrix multiplication) =
tensor([[0.7257, 0.8695, 0.6053, 0.5079, 0.3544],
        [1.4190, 1.2206, 0.8879, 0.8999, 0.4098],
        [1.3675, 2.0075, 1.1298, 1.0775, 0.8964],
        [1.8808, 2.0925, 2.0276, 1.4787, 1.3018],
        [1.7765, 2.0490, 1.8032, 1.3799, 1.1771]])
```

**图 3:** 2.1 的 $A+B$、$A-B$、逐元素乘积 $A*B$ 与矩阵乘积 $A@B$（均已明确标注）

### 3.2 2.2 形状与索引：reshape 与 permute

```
T = torch.arange(24).reshape(6, 4)   # shape (6, 4)
flat = T.reshape(-1)                 # shape (24,)
central = T[1:5, 1:3]                # rows 1..4, cols 1..2 -> (4, 2)
```

零基索引下，第 1 到第 4 行对应 `1:5`，第 1 到第 2 列对应 `1:3`，得到的中心切片形状为 $(4,2)$，值为

$$\begin{bmatrix} 5 & 6 \\ 9 & 10 \\ 13 & 14 \\ 17 & 18 \end{bmatrix}.$$

展平后形状为 $(24,)$，值为 $0,1,\ldots,23$；原张量为 $(6,4)$。对应截图见图 4、图 5。

**reshape 与 permute 的区别：**`reshape` 只改变张量的逻辑形状，按行优先（row-major）顺序重新解释同一串元素，不改变元素之间的先后次序；当新形状与原形状兼容且张量连续时返回视图，否则会复制数据。例如把 $(6,4)$ 变为 $(4,6)$ 时，元素仍按 $0,1,2,\ldots$ 的顺序逐行填入，所以各行是连续的 6 个数。`permute` 则是**交换/重排维度轴**：元素值保持原位置，但轴顺序改变，等价于转置（`T.permute(1,0)` 即 $T^\top$，形状 $(4,6)$，第 $i$ 列就是原来的第 $i$ 行）。permute 返回的是非连续的视图，通常无需复制，但后续某些需要连续内存的操作前要调用 `.contiguous()`。简单说：reshape 变“形状”，permute 变“轴的顺序”。

```
================================================================================
Question 2.2: shape and indexing
================================================================================
T = torch.arange(24).reshape(6, 4), shape=(6, 4)
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]])

Flattened T: T.reshape(-1), shape=(24,)
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23])

Central slice T[1:5, 1:3] (rows 1..4, columns 1..2), shape=(4, 2)
tensor([[ 5,  6],
        [ 9, 10],
        [13, 14],
        [17, 18]])
```

**图 4:** 2.2 的张量 $T$、展平结果与中心切片 $T[1:5, 1:3]$（形状 $(4, 2)$）

```
For comparison (same values, different index order):
  T.reshape(4, 6) reinterprets the row-major memory order, shape=(4, 6):
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23]])

  T.permute(1, 0) swaps the two axes without changing the values, shape=(4, 6):
tensor([[ 0,  4,  8, 12, 16, 20],
        [ 1,  5,  9, 13, 17, 21],
        [ 2,  6, 10, 14, 18, 22],
        [ 3,  7, 11, 15, 19, 23]])

All checks passed.
```

**图 5:** 2.2 中用于对比的 `reshape(4,6)` 与 `permute(1,0)`

## 4 自动求导与梯度累积（Question 3，0.75 分）

设 $X \in \mathbb{R}^{3\times 3}$ 为 `requires_grad=True` 的叶子张量，

$$M = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix}, \qquad L(X) = \sum_{i,j} X_{ij} M_{ij}.$$

由于 $L$ 对每个 $X_{ij}$ 是线性的，解析梯度为

$$\frac{\partial L}{\partial X_{ij}} = M_{ij} \quad \Longrightarrow \quad \nabla_X L = M.$$

即预测 `X.grad` 应恰好等于 $M$。

## 4.1 (a) 第一次反向传播

前向计算 $L = \sum X \odot M$ 后调用 `backward()`，得到 `X.grad`：

$$\begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix} = M,$$

与解析结果一致（程序用 `torch.allclose` 校验通过）。本次运行的 $L \approx 2.398318$。截图见图 6。

```
torch.manual_seed(2026)
X = torch.rand(3, 3, dtype=torch.float32, requires_grad=True)
M = torch.tensor([[1., 0., 2.], [1., 1., 0.], [0., 3., 1.]])
L = (X * M).sum()
L.backward()
# X.grad == M
```

```
================================================================================
Question 3: setup (seed 2026)
================================================================================
X (leaf tensor, requires_grad=True) =
tensor([[0.1525, 0.2519, 0.7310],
        [0.0643, 0.0187, 0.4945],
        [0.7581, 0.0009, 0.6982]], requires_grad=True)

M (constant coefficient matrix) =
tensor([[1., 0., 2.],
        [1., 1., 0.],
        [0., 3., 1.]])

L(X) = sum_ij X_ij * M_ij, so the analytical gradient dL/dX is M.
X.requires_grad = True, X.is_leaf = True

================================================================================
(a) First backward pass: L = (X * M).sum()
================================================================================
L = sum(X * M) = 2.398318

X.grad after one backward (expected M) =
tensor([[1., 0., 2.],
        [1., 1., 0.],
        [0., 3., 1.]])

X.grad matches M (torch.allclose): True
```

**图 6:** (a) 第一次 `backward()` 后的 `X.grad`，等于 $M$

## 4.2 (b) 不清空梯度时的累积

`backward()` 之后原前向图已被释放，不能对同一张图再次反向传播，因此用同一个叶子 $X$ 重新构造一份前向图 $L_2 = \sum X \odot M$ 再调用 `backward()`。PyTorch 的 `.grad` 默认是**累加**的，不清空时新的梯度会加到旧值上，故

$$\texttt{X.grad} \to M + M = 2M = \begin{bmatrix} 2 & 0 & 4 \\ 2 & 2 & 0 \\ 0 & 6 & 2 \end{bmatrix}.$$

运行结果与预测一致（图 7）。

```
================================================================================
(b) Second backward pass without clearing: gradients accumulate
================================================================================
The graph of L was freed by the first backward(), so build a new
forward graph with the same leaf X.  X.grad is NOT cleared here.

L2 = sum(X * M) = 2.398318   (same value as L)

X.grad after the second backward (expected 2 * M) =
tensor([[2., 0., 4.],
        [2., 2., 0.],
        [0., 6., 2.]])

X.grad matches 2 * M (torch.allclose): True
```

**图 7:** (b) 同一 `X.grad` 在第二次反向传播后变为 $2M$（梯度累积）

### 4.3 (c) 重置梯度与训练中清零的原因

执行 `X.grad = None` 后重新计算一次 $L_3$ 并反向传播，梯度重新从零开始累加，结果又回到 $M$（图 8）。

正常训练每一步要清空梯度，是因为参数梯度默认跨反向传播累加：若不清零，第 $k$ 步的 `step()` 会使用前 $k$ 个 mini-batch 梯度之和，等效于不断放大学习率并把历史 batch 的信息混入当前更新，导致更新方向错误。因此标准训练循环在每次 `backward()` 之前调用 `optimizer.zero_grad()`（或把 `.grad` 置 `None`），使本次更新只依赖当前 mini-batch。

```
================================================================================
(c) Reset X.grad = None, then backward once more
================================================================================
X.grad was set to None; L3 = sum(X * M) = 2.398318

X.grad after resetting and one backward (expected M again) =
tensor([[1., 0., 2.],
        [1., 1., 0.],
        [0., 3., 1.]])

X.grad matches M (torch.allclose): True

Summary of the three gradient results (each should be exact):
  (a) X.grad = M      = [[1.0, 0.0, 2.0], [1.0, 1.0, 0.0], [0.0, 3.0, 1.0]]
  (b) X.grad = 2 * M  = [[2.0, 0.0, 4.0], [2.0, 2.0, 0.0], [0.0, 6.0, 2.0]]
  (c) X.grad = M      = [[1.0, 0.0, 2.0], [1.0, 1.0, 0.0], [0.0, 3.0, 1.0]]

All checks passed.
```

**图 8:** (c) 重置 `X.grad = None` 后再反向传播，恢复为 $M$

## 5 MNIST 两层 MLP 分类（Question 4，2.00 分）

本题在提供的 `HW2_PyTorch_Basics.ipynb` 中完成全部 TODO，并已按顺序执行全部单元格。训练使用官方 60,000 张训练图，最终评估使用官方 10,000 张测试图；测试结果只用于报告，没有

参与任何设置或 checkpoint 的选择。

### 5.1 4(a) 数据加载与预处理

预处理流水线对训练集和测试集完全一致：

1. `v2.ToImage()`：把 PIL 图像转换为 $(1, 28, 28)$ 的张量；
2. `v2.ToDtype(torch.float32, scale=True)`：转换为 float32 并线性缩放到 $[0, 1]$；
3. `v2.Normalize((0.5,), (0.5,))`：$x \mapsto (x - 0.5)/0.5$，得到 $[-1, 1]$ 区间。

创建训练集与测试集两个 `datasets.MNIST` 对象（分别令 `train=True` 与 `train=False`），传入同一个 `transform`，并设置 `download=not available`；其中 `available` 来自 `prepare_mnist(root)`，本地文件已就绪时为 `True`，因此不会下载。DataLoader 使用 `batch_size=64`、`num_workers=0`，仅训练集 `shuffle=True`，其 `generator` 为 `torch.Generator().manual_seed(2026)`；测试集保持顺序遍历。

```
def make_mnist_loaders(root="data"):
    available = prepare_mnist(root=root)
    transform = v2.Compose([
        v2.ToImage(),
        v2.ToDtype(torch.float32, scale=True),
        v2.Normalize((0.5,), (0.5,)),
    ])
    train_dataset = datasets.MNIST(root=root, train=True,
                                   transform=transform, download=not available)
    test_dataset = datasets.MNIST(root=root, train=False,
                                  transform=transform, download=not available)
    generator = torch.Generator().manual_seed(2026)
    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True,
                              num_workers=0, generator=generator)
    test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False,
                             num_workers=0)
    return train_loader, test_loader
```

**表 2:** 一个训练 batch 的属性（notebook 自检输出）

| 项目 | 数值 |
|---|---|
| 训练集 / 测试集样本数 | 60,000 / 10,000 |
| 图像 batch 形状与 dtype | $(64, 1, 28, 28)$，`torch.float32` |
| 标签 batch 形状与 dtype | $(64,)$，`torch.int64` |
| 像素值范围 | $[-1.0, 1.0]$ |
| 标签范围 | $0 \sim 9$（断言 `labels.min()>=0`、`labels.max()<=9`） |
| 每轮训练 batch 数 / 测试 batch 数 | 938 / 157（最后一批分别为 32、16 个样本） |

In [3]:
```
train_loader, test_loader = make_mnist_loaders()
assert len(train_loader.dataset) == 60000 and len(test_loader.dataset) == 10000
assert train_loader.dataset.train and not test_loader.dataset.train
assert train_loader.batch_size == test_loader.batch_size == 64
assert train_loader.num_workers == test_loader.num_workers == 0
assert isinstance(train_loader.sampler, torch.utils.data.RandomSampler)
assert isinstance(test_loader.sampler, torch.utils.data.SequentialSampler)
images, labels = next(iter(train_loader))
assert images.shape == (64, 1, 28, 28) and labels.shape == (64,)
assert images.dtype == torch.float32 and labels.dtype == torch.int64
assert images.min() >= -1 and images.max() <= 1
assert labels.min() >= 0 and labels.max() <= 9
print("Training/test examples:", len(train_loader.dataset), len(test_loader.dataset))
print("Images:", tuple(images.shape), images.dtype)
print("Labels:", tuple(labels.shape), labels.dtype)
print("Image range:", images.min().item(), images.max().item())
```

```
Training/test examples: 60000 10000
Images: (64, 1, 28, 28) torch.float32
Labels: (64,) torch.int64
Image range: -1.0 1.0
```

**图 9:** 4(a) 数据自检单元格及其输出

**Dataset 与 DataLoader 的区别：**`Dataset` 描述“单个样本如何取得”，需要实现 `__len__` 与 `__getitem__`，负责索引、读取图像、套用 transform、返回 `(image, label)`；它本身不知道 batch 的概念。`DataLoader` 是 `Dataset` 之上的迭代器/调度器，负责按 batch_size 组批、shuffle、用 collate 把样本堆叠成张量、并行预取以及与多进程协作。`num_workers=0` 表示不使用额外的子进程，所有 batch 都在主进程中同步加载（数据从 Dataset 取出、collate、送入模型都在同一进程完成），没有子进程启动和跨进程拷贝开销，因此在本作业这种小数据集与 Windows 环境下最直接；当 `num_workers>0` 时，主进程会启动工作子进程并行取数，可能掩盖 I/O 瓶颈，但会带来进程开销。

## 5.2 4(b) 模型定义

模型严格为“两个 Linear 层、一个隐藏层”，结构为

$$(1, 28, 28) \xrightarrow{\text{Flatten}} 784 \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{Linear}} 10.$$

实现中 `nn.Flatten()` 把每张图展平成 784 维，两个 `nn.Linear` 分别为 $784 \to 128$ 与 $128 \to 10$，中间接 ReLU；`forward` 返回形状 $(B, 10)$ 的原始 logits，不接 softmax（softmax 由 `CrossEntropyLoss` 内部完成）。参数量为 $784 \times 128 + 128 + 128 \times 10 + 10 = 101{,}770$。

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        return self.fc2(x)
```

```
In [5]: torch.manual_seed(2026)
        model = MLP()
        linear_layers = [m for m in model.modules() if isinstance(m, nn.Linear)]
        assert [(m.in_features, m.out_features) for m in linear_layers] == [(784, 128), (128, 10)]
        with torch.no_grad():
            logits = model(images[:7])
        assert logits.shape == (7, 10) and torch.isfinite(logits).all()
        print(model)
        print("Logit shape:", tuple(logits.shape))

        MLP(
          (flatten): Flatten(start_dim=1, end_dim=-1)
          (fc1): Linear(in_features=784, out_features=128, bias=True)
          (relu): ReLU()
          (fc2): Linear(in_features=128, out_features=10, bias=True)
        )
        Logit shape: (7, 10)
```

**图 10:** 4(b) 模型自检单元格：层结构输出与 logits 形状 $(7, 10)$

### 5.3 4(c) 损失、反向传播与训练

损失函数为 `nn.CrossEntropyLoss()`，优化器为 `torch.optim.SGD(model.parameters(), lr=0.1)`。`train_one_epoch` 的核心循环如下（每个 batch）：

```
model.train()
for images_batch, labels_batch in loader:
    optimizer.zero_grad()                  # 1. 清空上一步的梯度
    logits = model(images_batch)           # 2. 前向
    loss = loss_fn(logits, labels_batch)   # 3. 计算交叉熵
    loss.backward()                        # 4. 反向传播，计算梯度
    optimizer.step()                       # 5. 用梯度更新参数
    total_loss += loss.item() * labels_batch.size(0)
    total_correct += (logits.argmax(dim=1) == labels_batch).sum().item()
    total_examples += labels_batch.size(0)
```

**哪一行计算梯度、哪一行更新参数：**`loss.backward()` 依据计算图做反向传播，把 $\partial L/\partial\theta$ 累加到每个参数的 `.grad` 上，它不改变参数值；`optimizer.step()` 才读取这些梯度并按 SGD 规则 $\theta \leftarrow \theta - \eta\,\partial L/\partial\theta$ 修改参数。二者之间还有 `optimizer.zero_grad()`：因为 `.grad` 默认累加，不清零会让多个 batch 的梯度混在一起，所以每个 batch 开始时先清零。

训练 5 轮，每轮的训练损失与准确率采用“按样本数加权的均值”，即 $\sum_b \text{loss}_b \cdot B_b / \sum_b B_b$，准确率为累计正确预测数除以累计样本数（包含最后一批不满 64 的情形）。结果见表 3。

**表 3:** 每轮训练指标（参数更新过程中累计的预测）

| Epoch | 训练损失（加权均值） | 训练准确率 |
|---|---|---|
| 1 | 0.400884 | 87.46% |
| 2 | 0.178810 | 94.65% |
| 3 | 0.132231 | 96.02% |
| 4 | 0.105875 | 96.77% |
| 5 | 0.091024 | 97.18% |

### 5.4 4(d) 测试与整体评估

`evaluate` 使用 `model.eval()` 与 `torch.no_grad()`：前者关闭 Dropout 等训练期行为，后者不记录计算图；按 batch 累加 loss $\times B$ 与正确预测数，最后除以样本总数（含最后较小的 batch），得到整个 loader 上的平均损失与准确率，过程中不更新任何参数。

在 5 轮训练前后，用同一个 `evaluate` 在完整训练集（顺序、不 shuffle、batch 64）上各评估一次；训练结束后再对完整测试集评估一次。最终结果见表 4，训练与测试输出见图 11。

**表 4:** 最终评估结果

| 评估对象 | 损失 | 准确率 |
|---|---|---|
| 完整训练集（训练前，初始模型） | 2.326570 | – |
| 完整训练集（训练后，固定模型） | 0.089063 | 97.34% |
| 完整测试集（训练后，仅评估一次） | 0.106050 | 96.79% |

```
In [9]: # Reinitialize so rerunning this cell starts the same experiment.
        torch.manual_seed(2026)
        model = MLP()
        train_loader.generator.manual_seed(2026)
        loss_fn, optimizer = make_training_components(model)
        assert isinstance(loss_fn, nn.CrossEntropyLoss)
        assert isinstance(optimizer, torch.optim.SGD)
        assert all(group["lr"] == 0.1 for group in optimizer.param_groups)
        train_eval_loader = DataLoader(train_loader.dataset, batch_size=64,
                                       shuffle=False, num_workers=0)
        initial_train_loss, initial_train_accuracy = evaluate(model, train_eval_loader, loss_fn)
        history = []
        for epoch in range(5):
            train_loss, train_accuracy = train_one_epoch(model, train_loader, loss_fn, optimizer)
            assert torch.isfinite(torch.tensor(train_loss)) and 0 <= train_accuracy <= 1
            history.append({"epoch": epoch + 1, "loss": train_loss, "accuracy": train_accuracy})
            print(f"Epoch {epoch + 1}/5: train loss={train_loss:.6f}, accuracy={train_accuracy:.2%}")
        final_train_loss, final_train_accuracy = evaluate(model, train_eval_loader, loss_fn)
        test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
        assert len(history) == 5 and final_train_loss < initial_train_loss
        assert torch.isfinite(torch.tensor(test_loss)) and 0 <= test_accuracy <= 1
        print(f"Full training set: initial loss={initial_train_loss:.6f}, final loss={final_train_loss:.6f}")
        print(f"Full training set: final accuracy={final_train_accuracy:.2%}")
        print(f"Final test set: loss={test_loss:.6f}, accuracy={test_accuracy:.2%}")

        Epoch 1/5: train loss=0.400884, accuracy=87.46%
        Epoch 2/5: train loss=0.178810, accuracy=94.65%
        Epoch 3/5: train loss=0.132231, accuracy=96.02%
        Epoch 4/5: train loss=0.105875, accuracy=96.77%
        Epoch 5/5: train loss=0.091024, accuracy=97.18%
        Full training set: initial loss=2.326570, final loss=0.089063
        Full training set: final accuracy=97.34%
        Final test set: loss=0.106050, accuracy=96.79%
```

**图 11:** 4(c)–(d) 训练与最终评估单元格及其输出

### 5.5 4(e) 保存与重载

保存时只写 `model.state_dict()` 到 `mnist_mlp.pt`（本机运行得到的文件大小为 409,585 字节），而不是序列化整个模型对象。重载时新建同结构的 `MLP`，用 `torch.load(path, map_location="cpu", weights_only=True)` 载入权重并 `load_state_dict`，再调用 `eval()`。其中 `map_location="cpu"` 保证权重在 CPU 上恢复；`weights_only=True` 只反序列化张量等安全

类型，避免加载任意 Python 对象，安全性更好。

```
def save_and_reload(model, path="mnist_mlp.pt"):
    torch.save(model.state_dict(), path)
    restored_model = MLP()
    state_dict = torch.load(path, map_location="cpu", weights_only=True)
    restored_model.load_state_dict(state_dict)
    restored_model.eval()
    return restored_model
```

随后在固定图像 batch `images[:16]` 上，用 `torch.no_grad()` 分别计算原模型与重载模型的 logits，并用 `torch.testing.assert_close` 校验完全一致；重载模型预测的数字为

$$[6, 2, 9, 1, 4, 2, 1, 0, 9, 7, 7, 8, 3, 9, 2, 2],$$

与原模型相同，重载检查通过（图 12）。

```
In [11]: restored = save_and_reload(model)
         assert restored is not model and not restored.training
         probe = images[:16].clone()  # Fixed images retained from the data self-check.
         model.eval()
         with torch.no_grad():
             expected, actual = model(probe), restored(probe)
         torch.testing.assert_close(actual, expected)
         print("Predicted digits:", actual.argmax(dim=1).tolist())
         print("Reload check passed: logits match on the same fixed image batch.")

         Predicted digits: [6, 2, 9, 1, 4, 2, 1, 0, 9, 7, 7, 8, 3, 9, 2, 2]
         Reload check passed: logits match on the same fixed image batch.
```

**图 12:** 4(e) 保存与重载检查单元格及其输出

### 5.6 结果分析

- 初始模型在训练集上的交叉熵为 2.3266，接近 10 类均匀随机时的 $\ln 10 \approx 2.3026$，符合随机初始化、输出接近均匀的预期；它也远大于训练后的 0.0891，说明训练确实显著降低了训练损失（notebook 中的断言 `final_train_loss` < `initial_train_loss` 也验证了这一点）。
- 5 轮内训练损失单调下降（0.4009 → 0.0910），每轮训练准确率从 87.46% 升到 97.18%，训练稳定、没有发散；对 784–128–10 的小 MLP，SGD 学习率 0.1 在 5 轮内已经收敛到较好的水平。
- 测试准确率 96.79%，训练（固定模型）准确率 97.34%，两者只差约 0.55 个百分点，泛化差距很小，5 轮训练下没有明显的过拟合；测试损失 0.1061 略高于训练损失 0.0891 也是同一现象。
- 按题目要求，测试集只在 5 轮训练全部结束后评估一次，未用于选择学习率、轮数或 checkpoint，因此该结果是干净的泛化估计。

## 6 AI 使用声明（AI Statement）

本作业的代码和报告中的思考题答案全部由本人自主学习完成，实验报告在 DeepSeek 的辅助下完成，具体情况如下：

1. **用途。**AI 用于帮助我润色、增色我草拟好的答案、核对概念解释（如 `CrossEntropyLoss` 的输入、`eval()` 与 `no_grad()` 的区别、梯度累积），起草 Q1.3、Q2、Q3 的脚本以及 Q4 notebook 中 TODO 的实现代码，并协助报告的文字组织与 LaTeX 排版。报告中的实验数据不是由 AI 生成的。
2. **本人的检查。**所有代码都在本地隔离环境（Python 3.11.15、PyTorch 2.12.1+cpu、TorchVision 0.27.1+cpu、CPU）中实际运行；notebook 的全部断言通过，报告中每个损失、准确率、梯度矩阵与截图都来自本地运行输出；我自行编写、逐项阅读并理解提交的代码，能够解释每个函数与关键语句的作用。
3. **局限与责任。**AI 输出可能包含错误或过时信息，若与官方文档或实际运行结果不一致，以实际运行结果为准；最终提交内容由本人负责。

## 7 复现说明与提交文件

本地复现步骤：安装题目指定版本区间的 PyTorch/TorchVision，在 `HW2` 目录下放置 MNIST 的四个 IDX 文件（或直接使用 `data/MNIST/raw` 中已准备好的文件），打开 `HW2_PyTorch_Basics.ipynb` 并从头到尾依次运行；Q1–Q3 的三个 Python 脚本可独立运行。

提交文件清单：

- `HW2_PyTorch_Basics.ipynb`：已执行、含全部输出；
- `mnist_mlp.pt`：本机训练得到的 checkpoint；
- 本报告 PDF；
- `hw2_q1_concept.py`、`hw2_q2_tensors.py`、`hw2_q3_autograd.py`：Q1.3 与 Q2、Q3 的独立代码；
- `prepare_mnist.py`：数据准备脚本。

未提交数据集文件与虚拟环境目录。

## 参考资料


- PyTorch 文档：`torch.nn.CrossEntropyLoss`，https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html
- PyTorch 文档：Autograd mechanics（`requires_grad`、`backward`、`retain_grad`），https://pytorch.org/docs/stable/notes/autograd.html
- PyTorch 文档：`torch.utils.data.DataLoader` 与 `Dataset`，https://pytorch.org/docs/stable/data.html
- TorchVision 文档：MNIST 数据集与 `transforms.v2`，https://pytorch.org/vision/stable/datasets.html